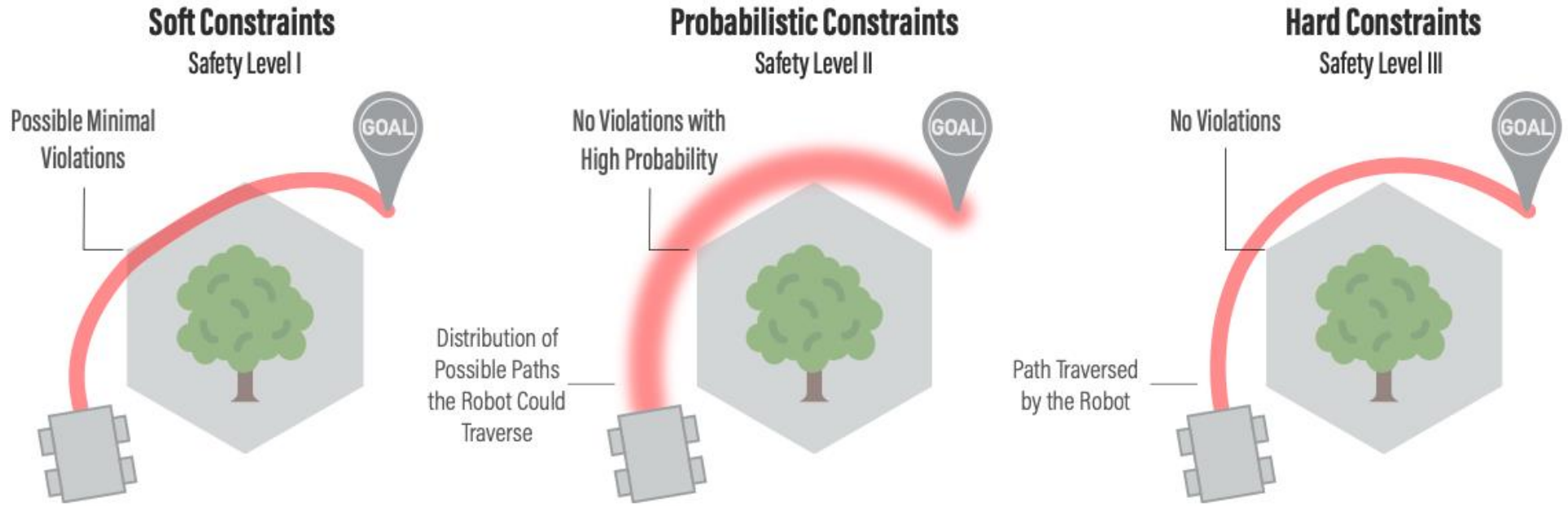
16-831: Introduction to Robot Learning, Fall 2024

Safe Robot Learning



Instructor: Guanya Shi

LeCAR
Learning and Control for Agile Robotics Lab

Carnegie Mellon University
The Robotics Institute

Recording Reminder

- This session will be audio recorded for educational use by other students in this course.

Guest Lectures

- Please attend in person. “Almost” mandatory (strongly encourage)!

Lecture 22, 11/19 (Tue)
Yonatan Bisk, CMU LTI

Foundation Models in Robotics

Logistics: Project

- ❑ 12/03 (Tue): project presentation I
- ❑ 12/05 (Thu): project presentation II
- ❑ 12/13: Project report due (up to 8 pages (including references), CoRL template)
- ❑ Detailed schedule and guidelines will be released soon

- ❑ Grading: 20% presentation + 20% report

Course Structure

Latest schedule: https://16-831.github.io/fall24/lectures

- ❑ Topic group 1: Course introduction: What is robot learning and why?
- ❑ Topic group 2: Machine/Deep learning refresher
- ❑ Topic group 3: Imitation learning
- ❑ Topic group 4: Model-free reinforcement learning
- ❑ Topic group 5: Model-based reinforcement learning
- ❑ Topic group 6: Bandit, preference-based learning, exploration
- ❑ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ❑ Topic group 8: Specialized topics (one guest lecture)
- ❑ Project presentation

Outline

- ❑ What is safety in robot learning?
  - Policy-wise
  - State-wise
- ❑ How to achieve/guarantee safety?
  - MBRL with stability guarantees
  - MBRL with state-wise safety guarantees
  - RL encouraging safety: CMDP framework

Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning

Lukas Brunke*, Melissa Greeff*, Adam W. Hall*, Zhaocong Yuan*, Siqi Zhou*, Jacopo Panerati, and Angela P. Schoellig

*Equal contribution
Institute for Aerospace Studies, University of Toronto, Toronto, Ontario, Canada, M3H 5T6; University of Toronto Robotics Institute, Toronto, Ontario, Canada, M5S 1A4; Vector Institute for Artificial Intelligence, Toronto, Ontario, Canada, M5G 1M1; emails: {firstname.lastname}@robotics.utias.utoronto.ca

Outline

- ❑ What is safety in robot learning?
  - Policy-wise
  - State-wise
- ❑ How to achieve/guarantee safety?
  - MBRL with stability guarantees
  - MBRL with state-wise safety guarantees
  - RL encouraging safety: CMDP framework

Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning

Lukas Brunke*, Melissa Greeff*, Adam W. Hall*, Zhaocong Yuan*, Siqi Zhou*, Jacopo Panerati, and Angela P. Schoellig

*Equal contribution
Institute for Aerospace Studies, University of Toronto, Toronto, Ontario, Canada, M3H 5T6; University of Toronto Robotics Institute, Toronto, Ontario, Canada, M5S 1A4; Vector Institute for Artificial Intelligence, Toronto, Ontario, Canada, M5G 1M1; emails: {firstname.lastname}@robotics.utias.utoronto.ca

What is “Safety” in Robot Learning?

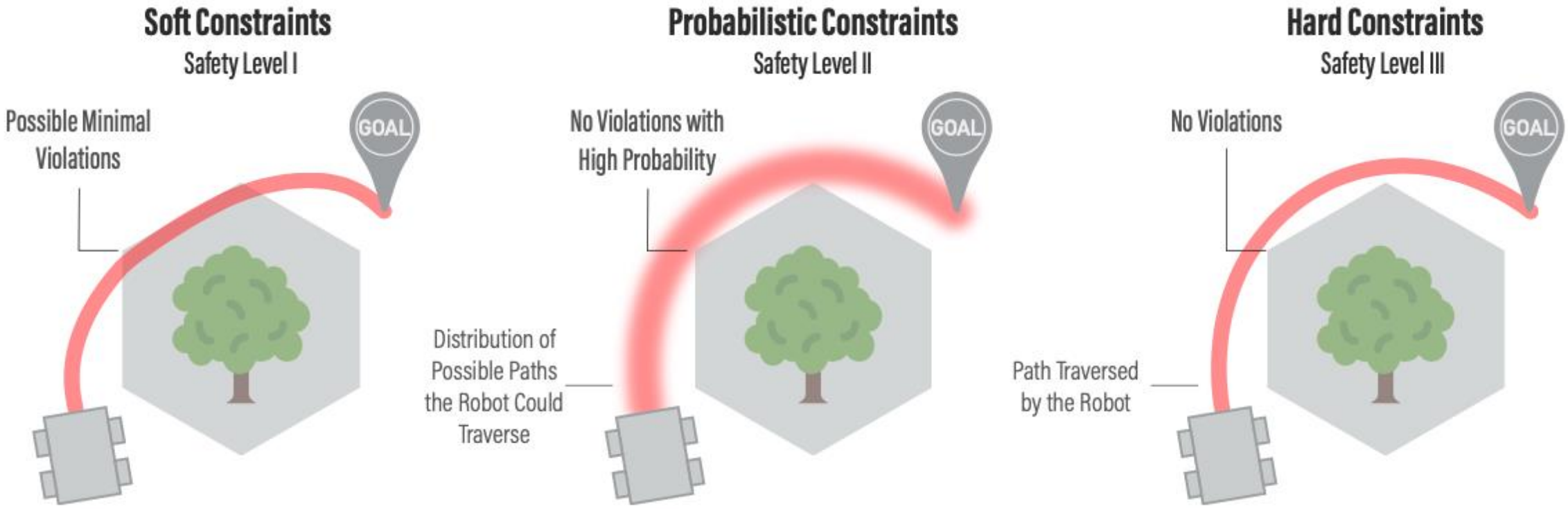
- ❑ Safety is essentially defined by **constraints** in an MDP
- ❑ Two main types of constraints:
  - Policy-wise (e.g., closed-loop stability, $\pi \in \Pi$)
  - State-wise (e.g., collision avoidance)

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases}$$

Three Different Levels

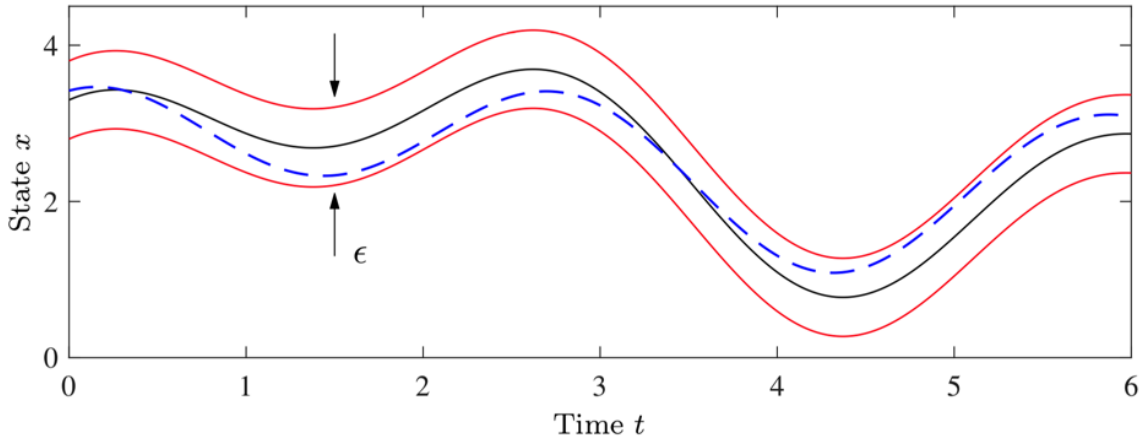
- Three different levels for constraint satisfaction:
  - Level I: encourage (e.g., Lagrangian-based method in safe RL)
  - Level II: probabilistic guarantees (e.g., chance constraints in optimal control)
  - Level III: hard constraints



Policy-wise Safety: Stability

- Given the dynamics model $\dot{x} = f(x, u)$
  - $x \in \mathbb{R}^n$: state of the robot; $u \in \mathbb{R}^m$: control input
  - Continuous-time version of $x_{t+1} = f(x_t, u_t)$
  - Policy: $u = \pi(x)$
  - Closed-loop system: $\dot{x} = f\big(x, \pi(x)\big) = f_{CL}(x)$

- Why study stability: If we perturb the robot a little bit, will the robot fall/fail?
- Examples: the black line is the "nominal behavior". A small drift in the beginning will NOT lead to divergence. The perturbed trajectory (blue dashed) stays in the red tube.



Astrom and Murray, Feedback Systems

Policy-wise Safety: Stability

- Given the dynamics model $\dot{x} = f(x, u)$
  - Closed-loop system: $\dot{x} = f\big(x, \pi(x)\big) = f_{CL}(x)$

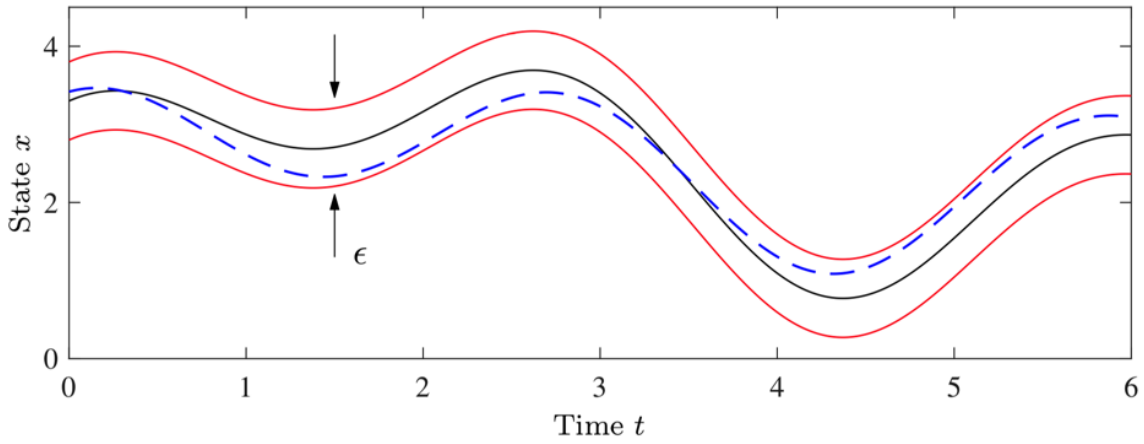
Definition (stable)

Let $x(t, a)$ be a solution to $f_{CL}(x)$ with $x(0) = a$. A solution is *stable* if other solutions that start near a stay close to $x(t, a)$. For all $\epsilon > 0$, there exists a $\delta > 0$ such that

$$\forall b, \|b - a\| < \delta \implies \|x(t, b) - x(t, a)\| < \epsilon, \forall t$$

When $x = a$ is an equilibrium point ($x(t, a) \equiv a$):

$$\forall b, \|b - a\| < \delta \implies \|x(t, b) - a\| < \epsilon, \forall t$$



Astrom and Murray, Feedback Systems

Policy-wise Safety: Stability

Definition (asymptotically stable)

For all $\epsilon > 0$, there exists a $\delta > 0$ such that

$$\forall b, \|b - a\| < \delta \implies \|x(t,b) - x(t,a)\| < \epsilon, \forall t \text{ and}$$

$$\lim_{t \to \infty} \|x(t,b) - x(t,a)\| = 0$$

Definition (exponentially stable)

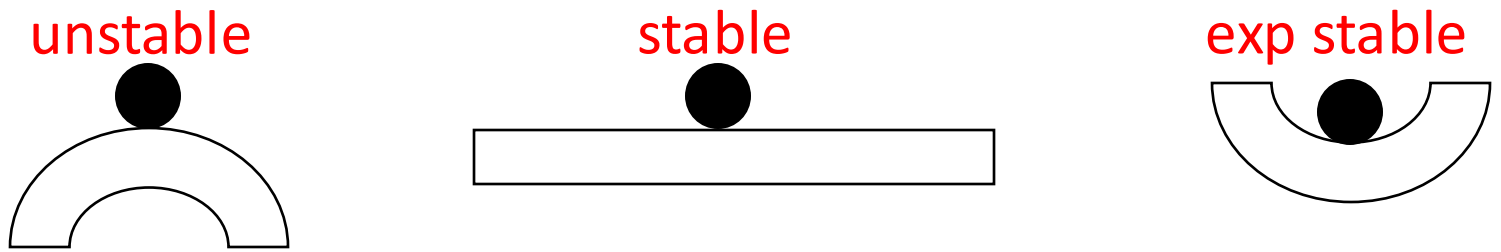
Asymptotically stable and

$$\|x(t,b) - x(t,a)\| < \alpha \|b - a\| e^{-\beta t}, \forall t$$

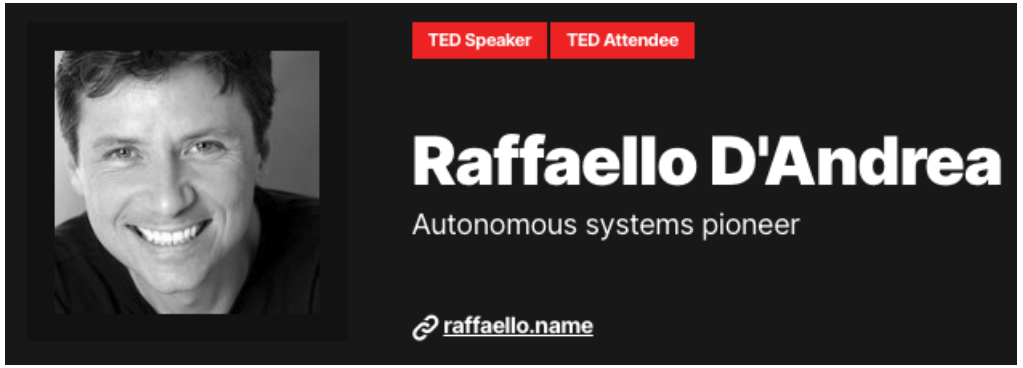
- Stable: $\|x(t,b) - x(t,a)\|$ is bounded
- Asymptotically stable: convergence
- Exponentially stable: fast (exponential) convergence

Policy-wise Safety: Stability

- Cartoonish example:



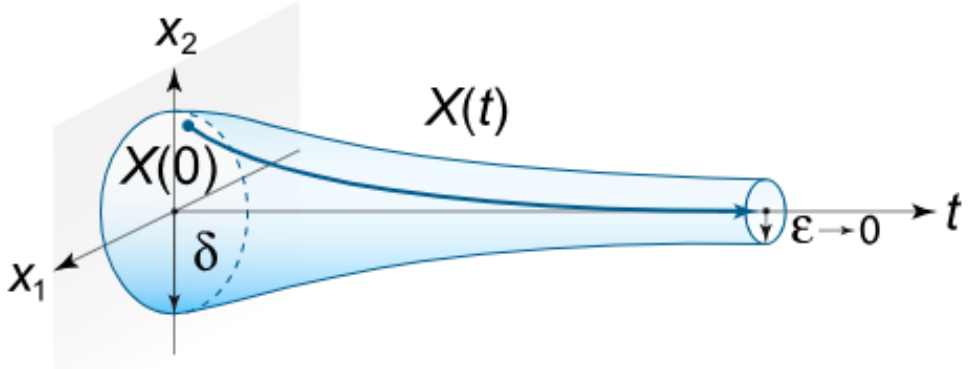
- Drone example:



Policy-wise Safety: Stability

- Linear system stability:
  - Consider a linear system $x_{t+1} = Ax_t + Bu_t$
  - Linear policy $u_t = -Kx_t$
  - The closed-loop system is $x_{t+1} = (A - BK)x_t$

- Stable if and only if: $\rho(A - BK) \leq 1$
  - $\rho(A - BK)$ is the spectrum radius of $A - BK$ (maximum of the absolute values of its eigenvalues)
- Exponentially stable if and only if: $\rho(A - BK) < 1$
- Why? $x_t = (A - BK)^t x_0$



State-wise Safety

- Constraint $x_t \in X_t$

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases}$$

- Exist in almost ALL *real* robotic systems!

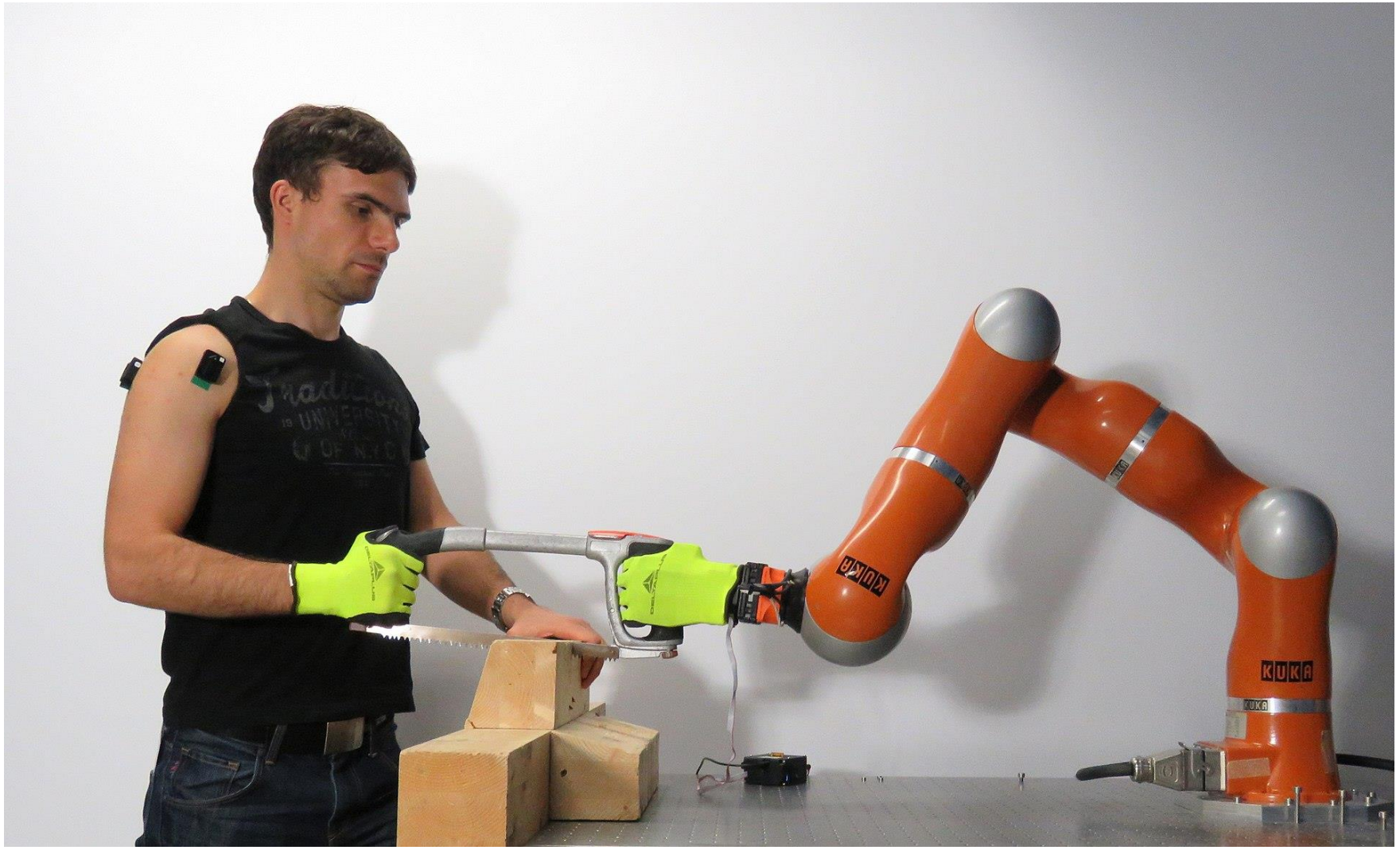
State-wise Safety

- Example: collision avoidance

https://agile-but-safe.github.io/

State-wise Safety

- Example: human robot interaction



https://en.wikipedia.org/wiki/Human%E2%80%93robot_collaboration

State-wise Safety

- Example: autonomous vehicles

https://discover.rbcroyalbank.com/self-driving-cars-put-safety-first/

Outline

- ❑ What is safety in robot learning?
  - Policy-wise
  - State-wise
- ❑ How to achieve/guarantee safety?
  - MBRL with stability guarantees
  - MBRL with state-wise safety guarantees
  - RL encouraging safety: CMDP framework

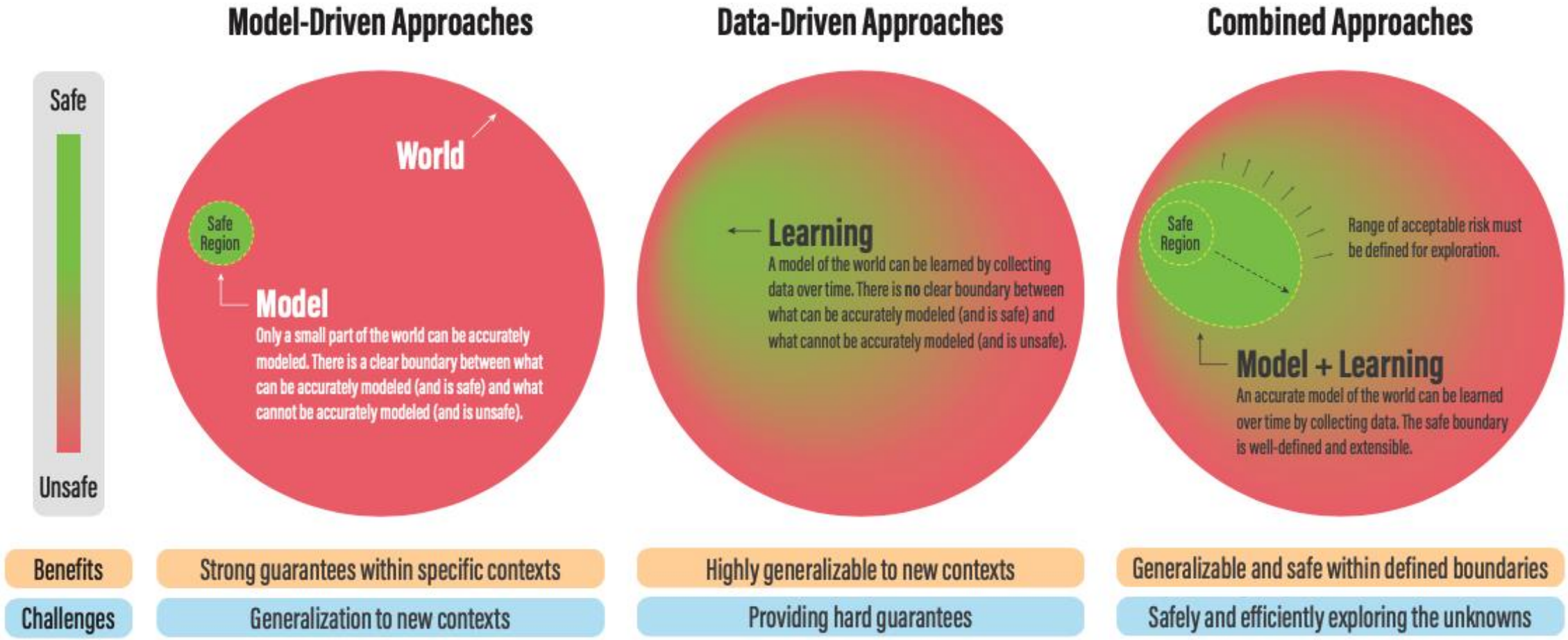
Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning

Lukas Brunke*, Melissa Greeff*, Adam W. Hall*, Zhaocong Yuan*, Siqi Zhou*, Jacopo Panerati, and Angela P. Schoellig

*Equal contribution
Institute for Aerospace Studies, University of Toronto, Toronto, Ontario, Canada, M3H 5T6; University of Toronto Robotics Institute, Toronto, Ontario, Canada, M5S 1A4; Vector Institute for Artificial Intelligence, Toronto, Ontario, Canada, M5G 1M1; emails: {firstname.lastname}@robotics.utias.utoronto.ca

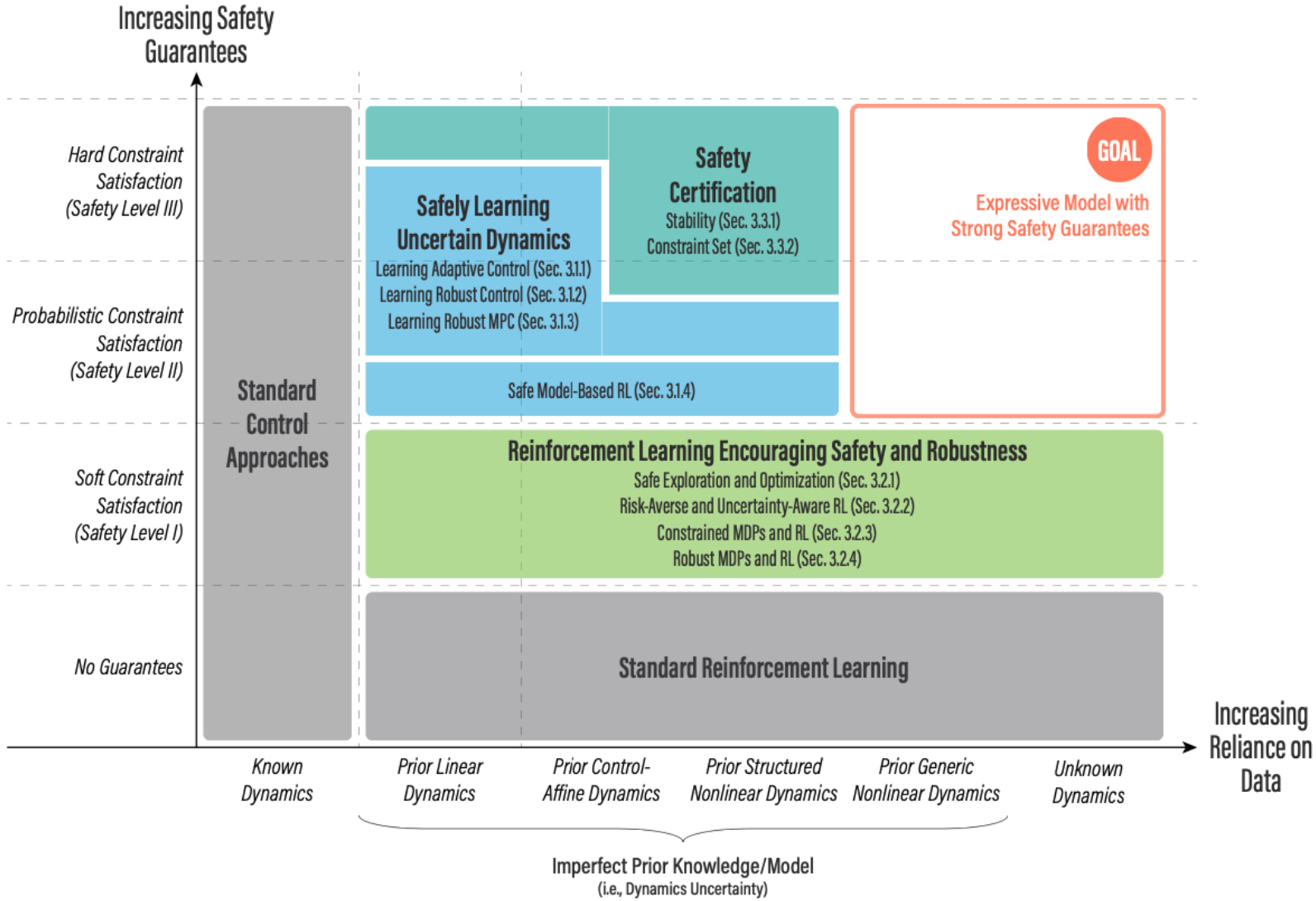
How to Achieve/Guarantee Safety?

- ❑ Tradeoffs between safety and ”generalizability”
- ❑ Combined approaches are promising



How to Achieve/Guarantee Safety?

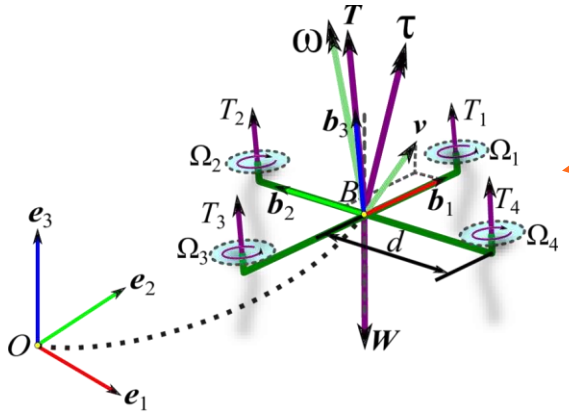
- ❑ An overview:
- ❑ Today:
  - Safe model-based RL
  - RL encouraging safety and robustness
- ❑ Very diverse and active research filed! Today we will do some case study.



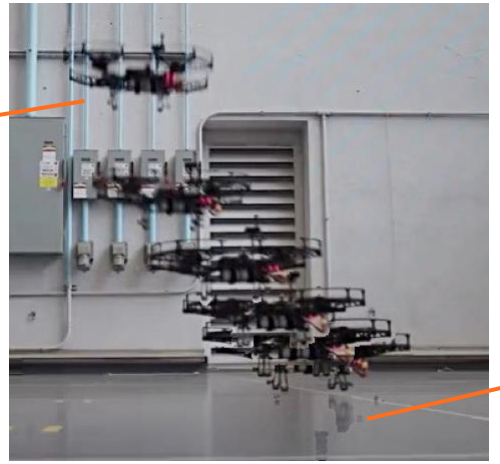
Model-based RL with Stability Guarantees: Neural-Lander

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = u + f(q,\dot{q},u)$$

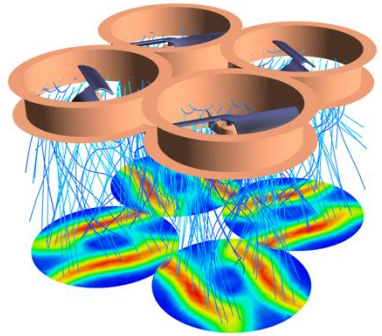
- $f(q,\dot{q},u)$ is the unknown aerodynamics depending on u
- Idea: use a DNN $\hat{f}(q,\dot{q},u)$ to approximate $f(q,\dot{q},u)$
- *Q: How to guarantee stability?*



easy to model and symbolic



Neural-Lander
[Shi et al., ICRA'19]



much harder to model!

Model-based RL with Stability Guarantees: Neural-Lander

Nonlinear tracking controller:

$$u = \text{linear feedback} + \text{nonlinear feedforward} - \hat{f}(q, \dot{q}, u_{\text{old}})$$

learned dynamics (a DNN)

Theorem [Shi et al., ICRA'19]

Suppose the Lipschitz of $\hat{f}$ is L. If $L < 1$, we have:

$$\|q - q_d\| \to C \cdot \frac{\epsilon}{\lambda_{\min}(K) - L\rho} \text{ exponentially}$$

approximation error ($\|f - \hat{f}\|_\infty$)

control gain

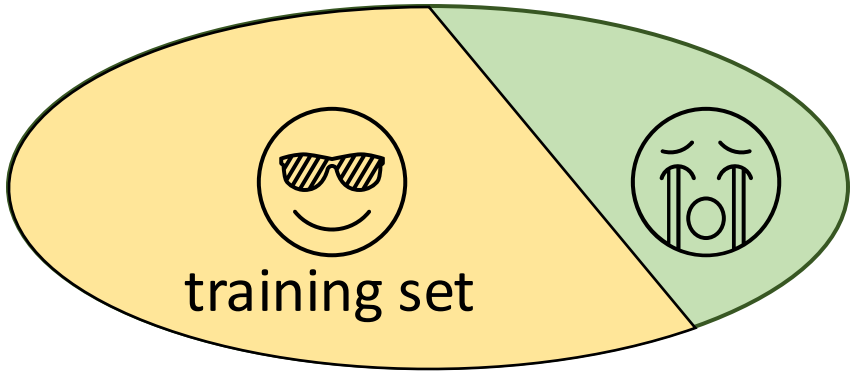
time delay

- In experiments, the drone crashed without this constraint!
- w/o constraint: $L \approx 500$
- w/ constraint (using spectral normalization): $L < 1$

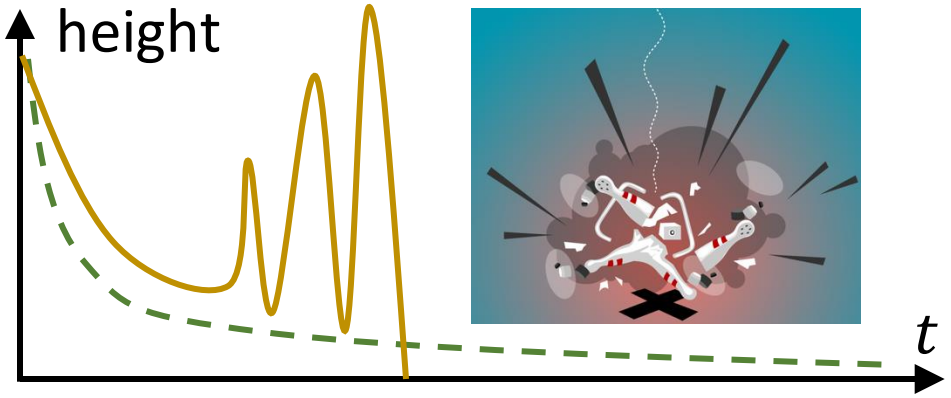
Model-based RL with Stability Guarantees: Neural-Lander

- Do we have to constraint the DNN? Yes! If we don't:

Learning perspective: $\hat{f}$ can not generalize

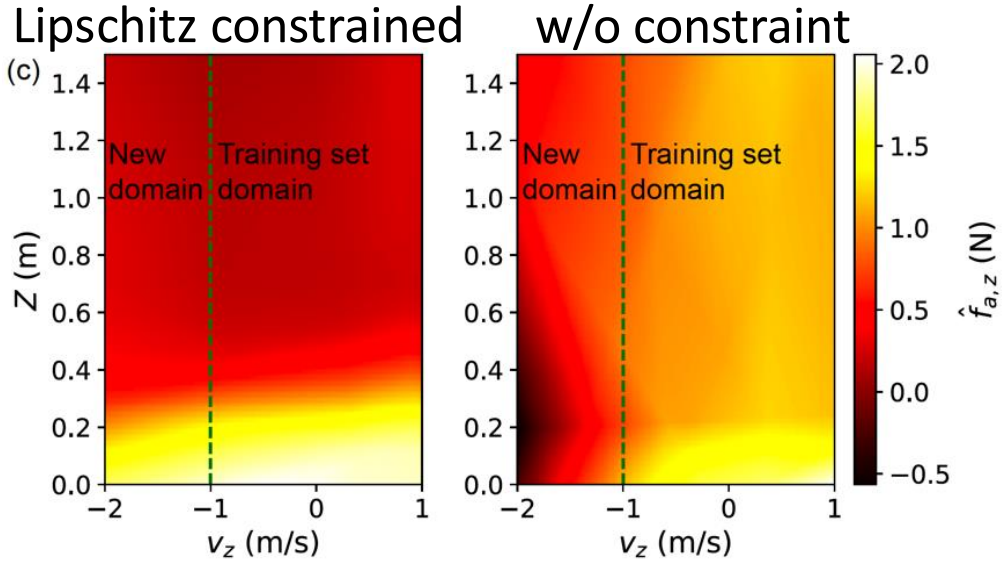


Control perspective: closed-loop instability



2D heatmaps of the learned $\hat{f}$

Lipschitz constrained w/o constraint



MBRL with Stability Guarantees: Lyapunov Learning

- Stability analysis in high-dimensional space is tricky (you don't want to solve ODEs in high-dimensional space and check its stability)
- Lyapunov function $V(x)$: $\mathbb{R}^n \to \mathbb{R}_{\geq 0}$ is an energy-like positive definite function that can provide *sufficient* condition for stability

Stability via Lyapunov function (informal)

$\dot{V} \leq 0$	$\dot{V} < 0$	$\dot{V} \leq -\alpha V$
stable	asymptotically stable	exponentially stable

- Example: single integrator system $\dot{x} = u$ ($x, u \in \mathbb{R}^n$) with a controller $u = -Kx$
  - Consider the Lyapunov function $V(x) = \frac{1}{2} x^\top x$
  - $\dot{V} = x^\top \dot{x} = -x^\top K x$
  - $\dot{V} \leq -2\lambda_{\min}(K) V$
  - Therefore, exponentially stable if K is positive definite!

MBRL with Stability Guarantees: Lyapunov Learning

- Key idea of Lyapunov learning:
  - Learn or design a Lyapunov certificate
  - Learn a policy to maximize the stability region

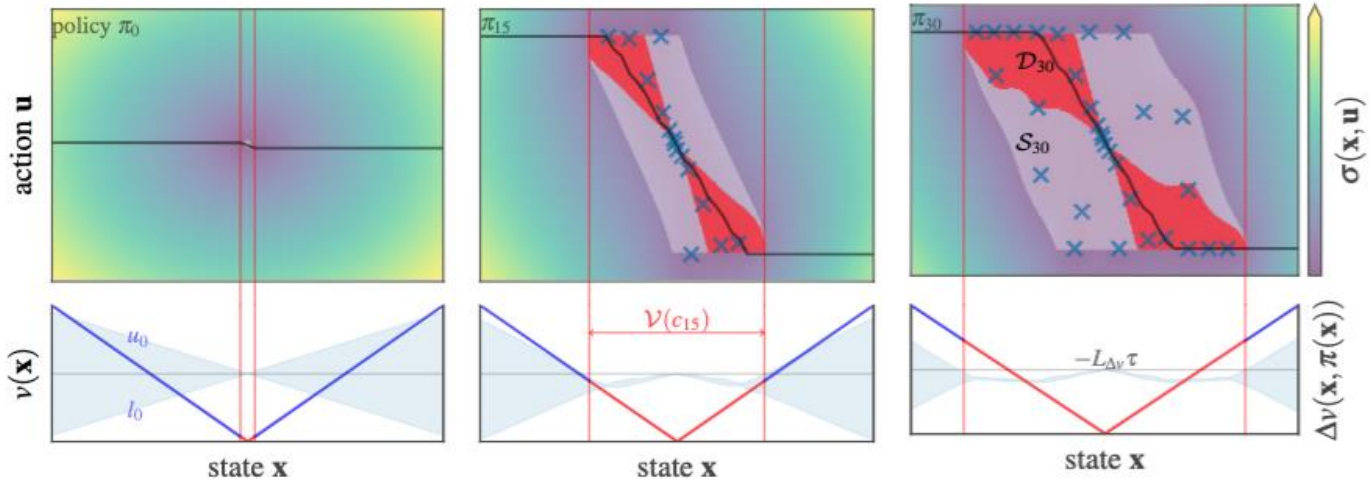
Safe Model-based Reinforcement Learning with Stability Guarantees

Felix Berkenkamp
Department of Computer Science
ETH Zurich
befelix@inf.ethz.ch

Matteo Turchetta
Department of Computer Science,
ETH Zurich
matteotu@inf.ethz.ch

Angela P. Schoellig
Institute for Aerospace Studies
University of Toronto
schoellig@utias.utoronto.ca

Andreas Krause
Department of Computer Science
ETH Zurich
krausea@ethz.ch



(a) Initial safe set (in red). (b) Exploration: 15 data points. (c) Final policy after 30 evaluations.

MBRL with Stability Guarantees: Lyapunov Learning

- Key idea of Lyapunov learning:
  - Learn or design a Lyapunov certificate
  - Learn a policy to maximize the stability region

Algorithm 1 SAFELYAPUNOVLEARNING

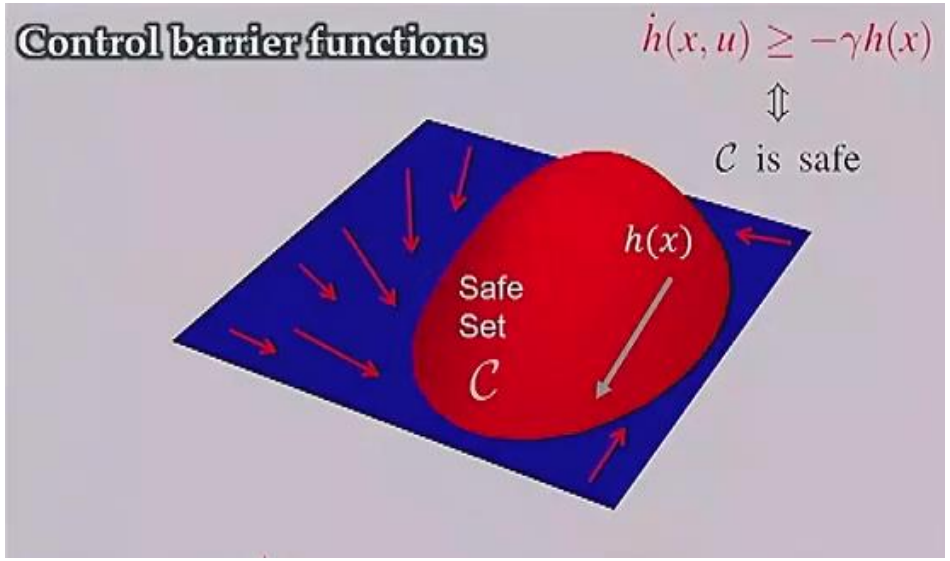
Input: Initial safe policy π_0, dynamics model $\mathcal{GP}(\mu(\mathbf{z}), k(\mathbf{z}, \mathbf{z}'))$
for all $n = 1, \ldots$ **do**
  Compute policy π_n via SGD on (7)
  $c_n = \operatorname{argmax}_c c$, such that $\forall \mathbf{x} \in \mathcal{V}(c_n) \cap \mathcal{X}_\tau : u_n(\mathbf{x}, \pi_n(\mathbf{x})) - v(\mathbf{x}) < -L_{\Delta v}\tau$
  $\mathcal{S}_n = \{(\mathbf{x}, \mathbf{u}) \in \mathcal{V}(c_n) \times \mathcal{U}_\tau \,|\, u_n(\mathbf{x}, \mathbf{u}) \leq c_n\}$
  Select $(\mathbf{x}_n, \mathbf{u}_n)$ within $\mathcal{S}_n$ using (6) and drive system there with backup policy π_n
  Update GP with measurements $f(\mathbf{x}_n, \mathbf{u}_n) + \epsilon_n$

$$\pi_n = \underset{\pi_\theta \in \Pi_L}{\operatorname{argmin}} \sum_{\mathbf{x} \in \mathcal{X}_\tau} r(\mathbf{x}, \pi_\theta(\mathbf{x})) + \gamma J_{\pi_\theta}(\mu_{n-1}(\mathbf{x}, \pi_\theta(\mathbf{x})) + \lambda \Big(u_n(\mathbf{x}, \pi_\theta(\mathbf{x})) - v(\mathbf{x}) + L_{\Delta v}\tau \Big), \quad (7)$$

MBRL with Safety Guarantees: Control Barrier Functions

- Safety function: $h(x) \geq 0$
- Control Barrier Function (CBF) provides a sufficient safety condition: If the initial state is safe and $\dot{h} \geq -\gamma h(x)$, the whole trajectory will be safe
  - Note that $\dot{h}$ depends on action: $\dot{h} = \frac{dh}{dx} f(x, u)$

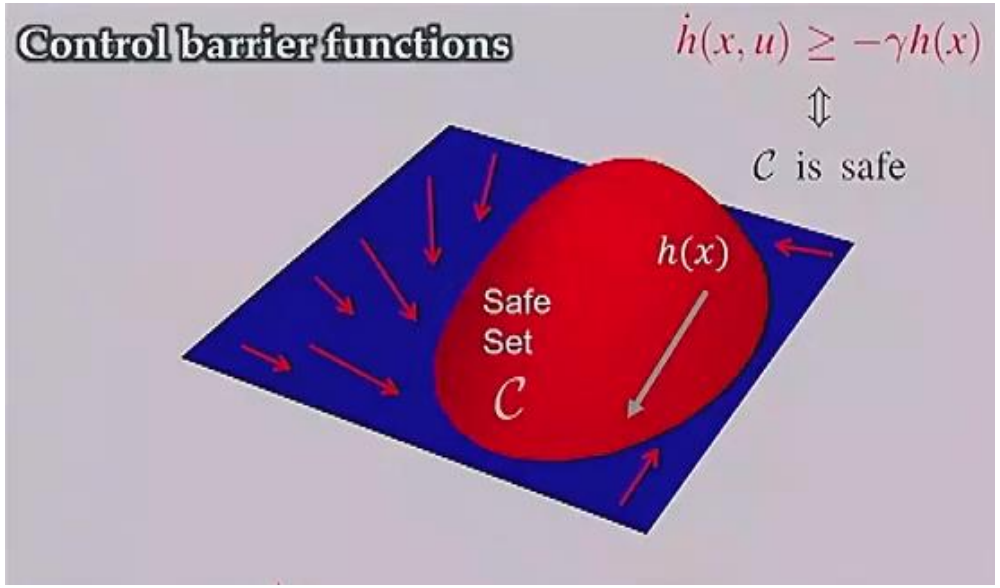
- Intuition: when $h(x)$ is large (very safe), $\dot{h}$ can be very negative; when $h(x)$ is small (close to the safety boundary), $\dot{h}$ has to be (near) positive



MBRL with Safety Guarantees: Control Barrier Functions

- Safety function: $h(x) \geq 0$
- Control Barrier Function (CBF) provides a safety condition: $\dot{h} \geq -\gamma h(x)$

- CBF-QP [Ames et al., 2017]: find the closest safe action to the nominal action (e.g., RL policy)
- In other words, it is a safety filter which projects unsafe action to be safe



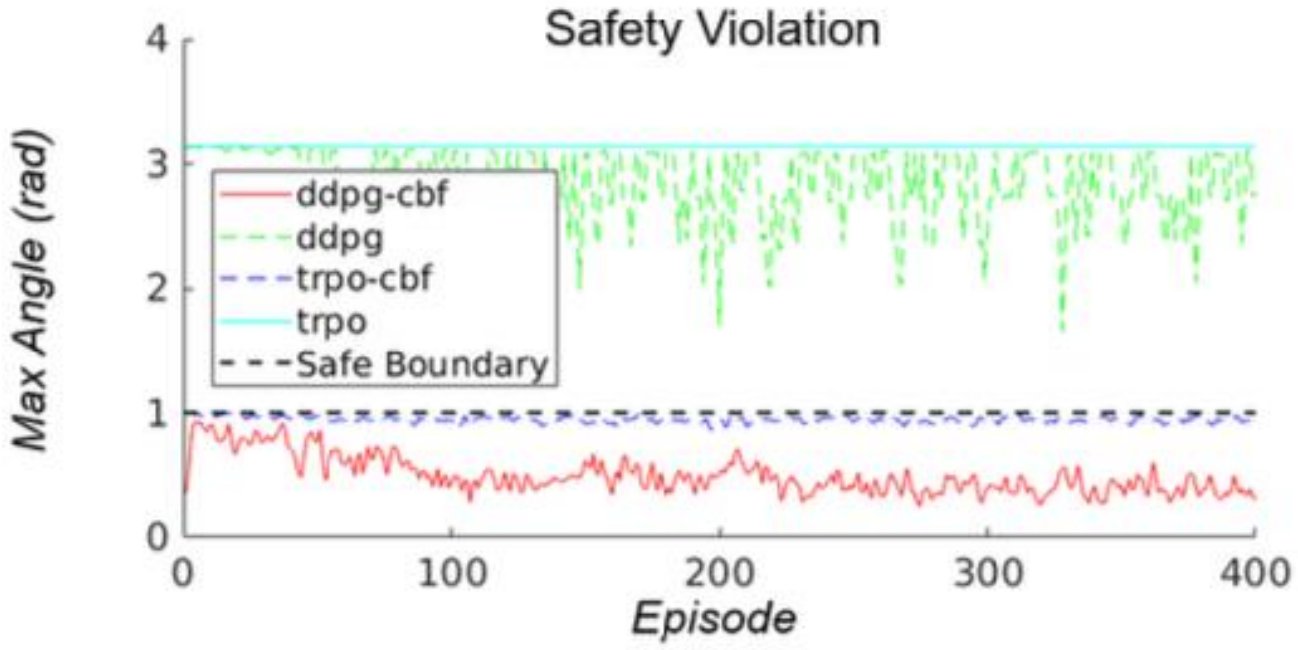
$$
\begin{aligned}
a_t^{\text{SAFE}} = \underset{a_t}{\text{argmin}} \quad & \|a_t - \pi_{\theta_\pi}(x_t, \hat{z}_t)\|_2 \\
\text{s.t.} \quad & p^T f_{\theta_f}(x_t, \hat{z}_t) + p^T g_{\theta_g}(x_t, \hat{z}_t)\, a_t + p^T q \\
& \geq (1-\eta) h(x_t) + \epsilon \\
& a_t \in \mathcal{A},
\end{aligned}
$$

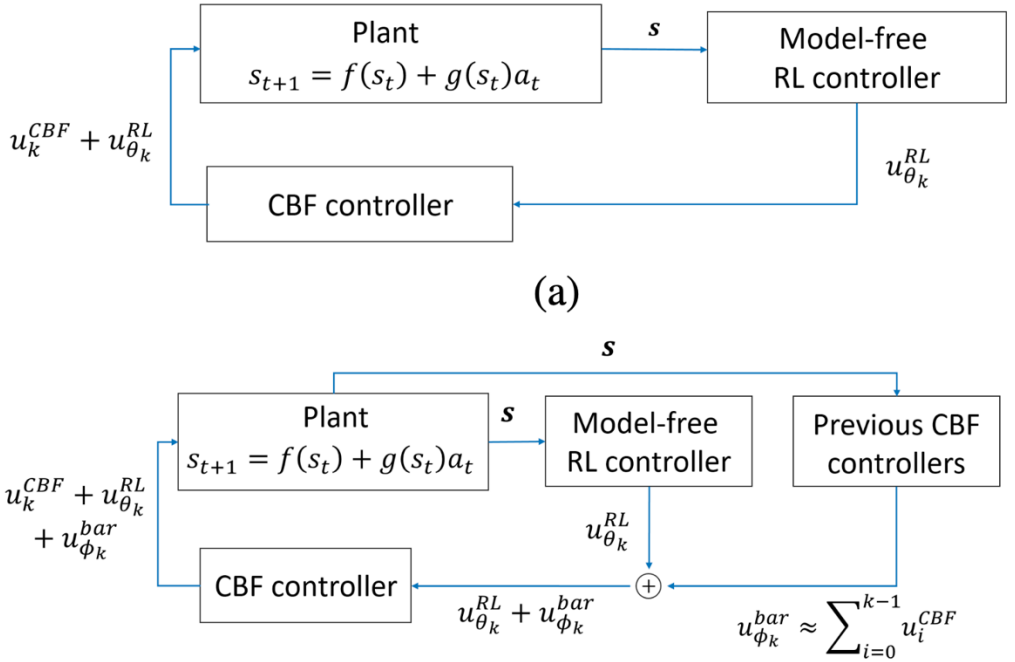
MBRL with Safety Guarantees: Control Barrier Functions

❑ Example 1:

End-to-End Safe Reinforcement Learning through Barrier Functions for Safety-Critical Continuous Control Tasks

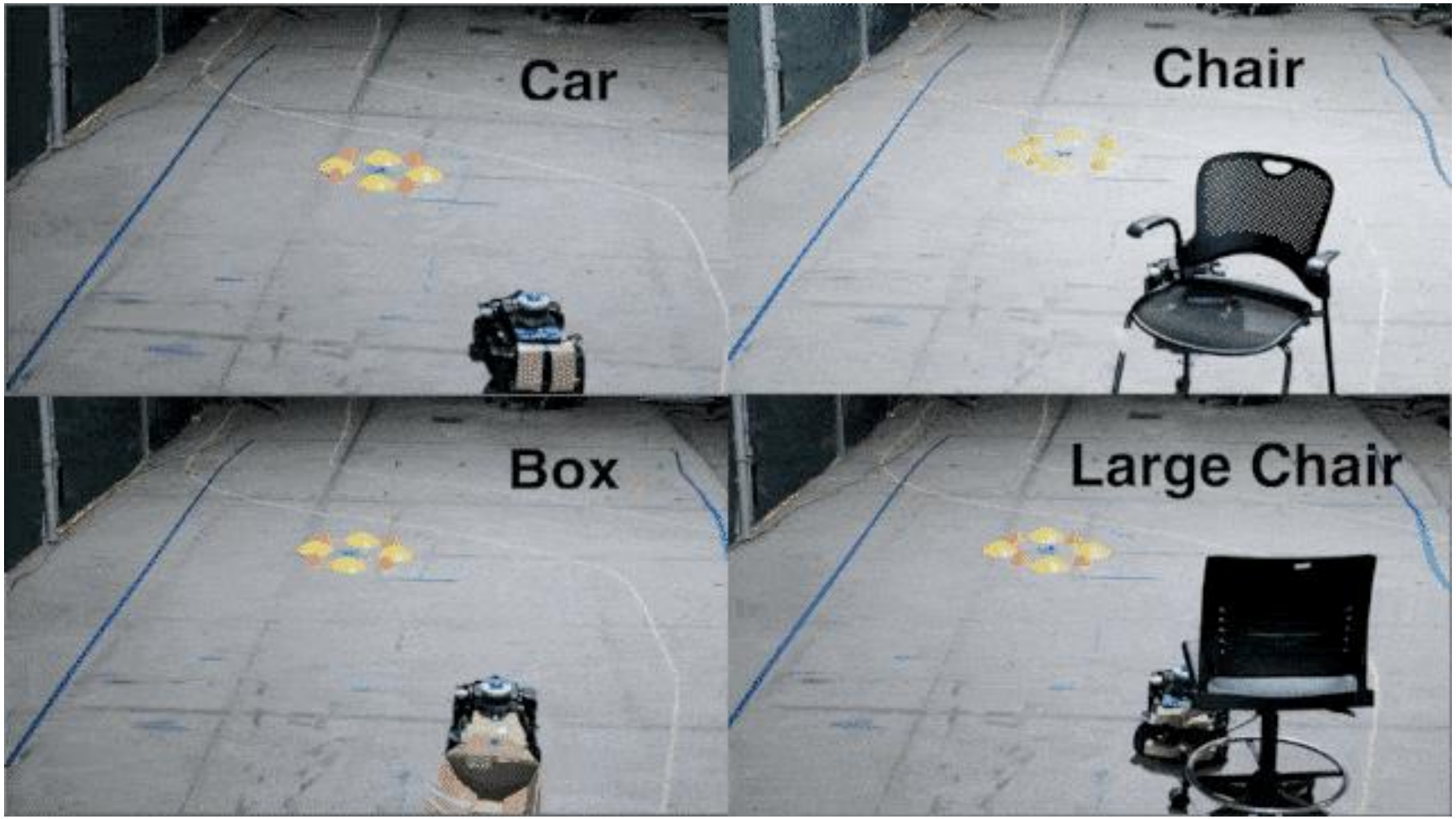
Richard Cheng,[1] Gábor Orosz,[2] Richard M. Murray,[1] Joel W. Burdick[1]
[1]California Institute of Technology, [2]University of Michigan, Ann Arbor





MBRL with Safety Guarantees: Control Barrier Functions

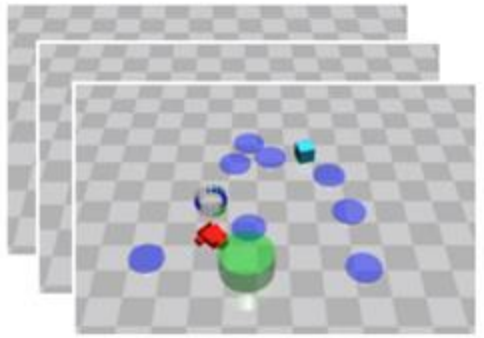
- Example 2: Safe Deep Policy Adaptation

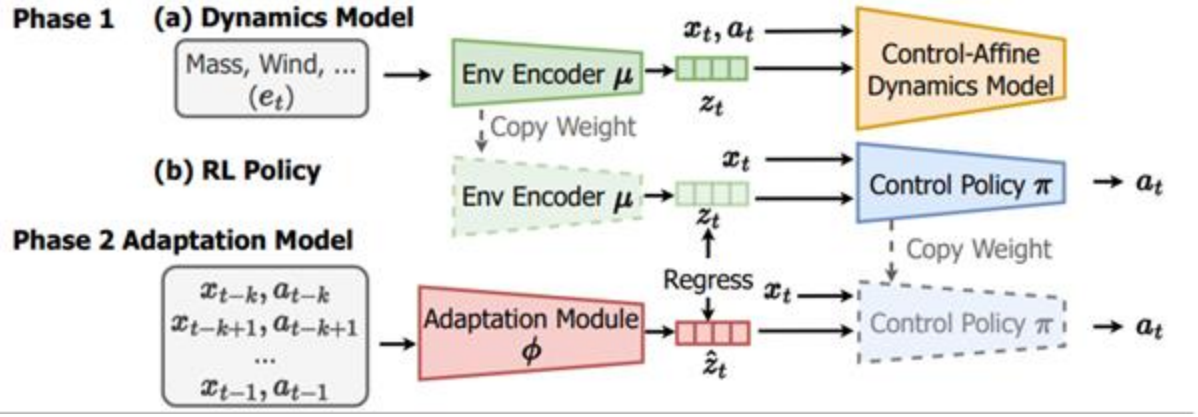


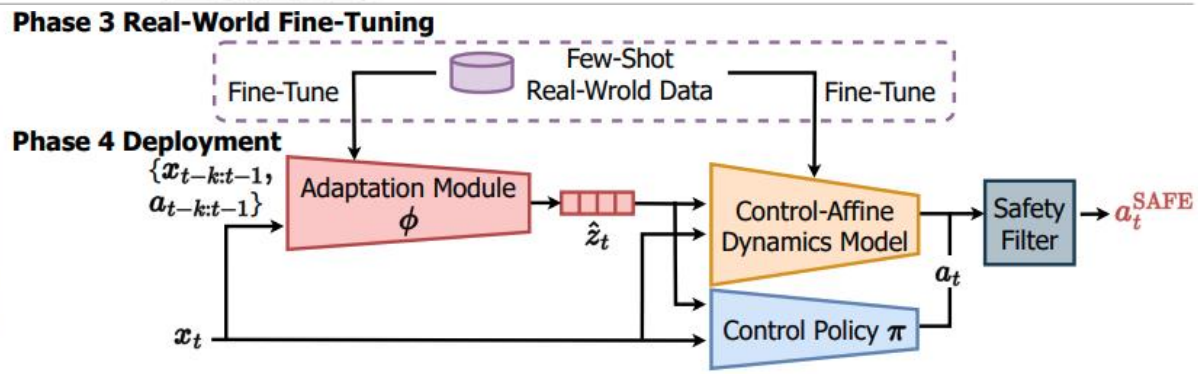
[Safe Deep Policy Adaptation, Xiao* and He* et al., ICRA'24]

MBRL with Safety Guarantees: Control Barrier Functions

- ❑ Phase 1 (sim): Learn *environment-conditioned* policy and dynamics model
- ❑ Phase 2 (sim): Learn the adaptation module (teacher-student framework)
- ❑ Phase 3 (real): fine-tune the model and adaptation module
- ❑ Phase 4 (deployment): deployment with the CBF-QP safety filter







[Safe Deep Policy Adaptation, Xiao* and He* et al., ICRA'24]

MBRL with Safety Guarantees: Other Methods

- ❑ Hamilton-Jacobian reachability
- ❑ Predictive methods
- ❑ Details: read this tutorial paper

MBRL with Safety Guarantees: Other Methods

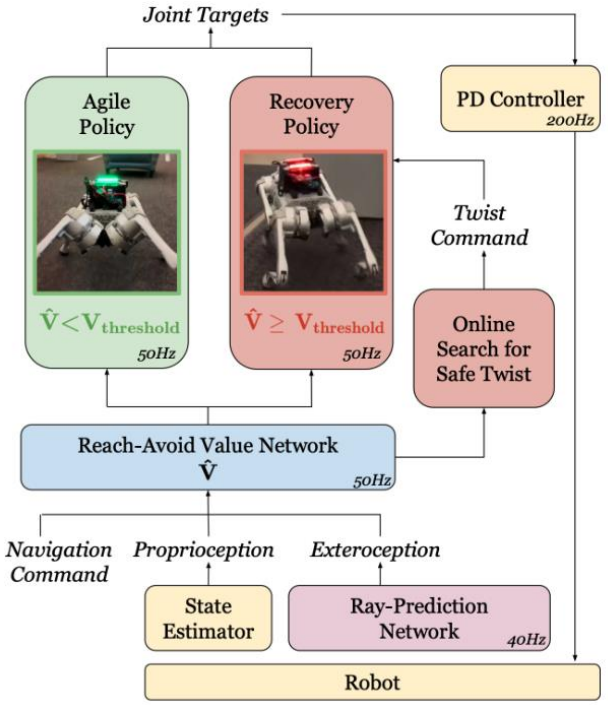
❑ Example of reachability-based safety

- Fully onboard & autonomous
- Fast (up to 3.1m/s)
- Safe (collision avoidance guarantees)
- Robust

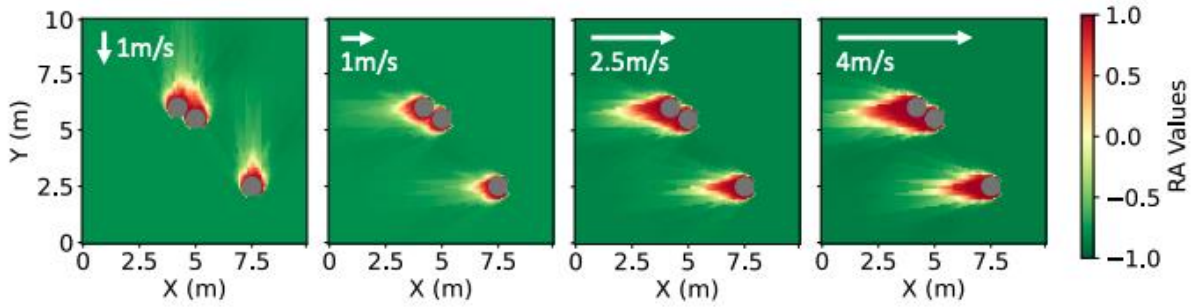
Agile But Safe: Learning Collision-Free High-Speed Legged Locomotion (https://agile-but-safe.github.io/)

MBRL with Safety Guarantees: Other Methods

- Our idea: RL + reach-avoid value network
  - The reach-avoid value $V_{\mathrm{RA}}^{\pi}(s) < 0$: π is safe and can reach the goal
  - $V_{\mathrm{RA}}^{\pi}(s) > 0$: π is unsafe or cannot reach the goal
- $V_{\mathrm{RA}}^{\pi}(s)$ can be efficiently learned via Bellman equation (policy evaluation)
  - $l(s) \leq 0$ when goal reaching; $\zeta(s) > 0$ when unsafe
- Learning $V_{\mathrm{RA}}^{\pi}(s)$ can be model-free and scalable!



$$V_{\mathrm{RA}}^{\pi}(s) = \gamma_{\mathrm{RA}} \max\left\{\zeta(s), \min\left\{l(s), V_{\mathrm{RA}}^{\pi}\left(f\left(s, \pi(s)\right)\right)\right\}\right\} + (1-\gamma_{\mathrm{RA}}) \max\left\{l(s), \zeta(s)\right\}.$$



RL Encouraging Safety: CMDP Framework

- Constrained MDP

$$\pi^* = \arg\max_{\pi \in \Pi_\theta} \; R(\pi)$$
$$\text{s.t. } C_i(\pi) \leq d_i, \; \forall i \in [m].$$

- Highly nonconvex, very hard to solve!
- Therefore, typically use Lagrangian methods:

$$\mathcal{L}(\pi, \lambda) = R(\pi) - \sum_i \lambda_i \Big(C_i(\pi) - d_i \Big),$$

- Duality:

$$(\pi^*, \lambda^*) = \arg\min_{\lambda \geq 0} \max_{\pi \in \Pi_\theta} \mathcal{L}(\pi, \lambda).$$

RL Encouraging Safety: CMDP Framework

- Constrained MDP

$$\pi^* = \arg\max_{\pi \in \Pi_\theta} \; R(\pi)$$
$$\text{s.t. } C_i(\pi) \leq d_i, \; \forall i \in [m].$$

- Therefore, typically use Lagrangian methods:

$$\mathcal{L}(\pi, \lambda) = R(\pi) - \sum_i \lambda_i \Big(C_i(\pi) - d_i \Big),$$

- Algorithms:

  - Fix $\lambda = \lambda^{(k)}$ and perform policy gradient update: $\theta_{k+1} = \theta_k + \alpha_k \nabla_\theta (\mathcal{L}(\pi(\theta), \lambda^{(k)}))|_{\theta=\theta_k}$, where α_k is the step size. The policy gradient could be on-policy likelihood ratio policy gradient (e.g., REINFORCE [15] and TRPO [5]) or off-policy deterministic policy gradient (e.g., DDPG [16]).

  - Fix $\pi = \pi_k$ and perform dual update $\lambda^{(k+1)} = f_k(\lambda^{(k)}, \pi_k)$. Existing methods for CMDPs, such as PDO and CPO, differ in the choice of the dual update procedure $f_k(\cdot)$. For example, PDO uses the simple dual gradient ascent $\lambda_i^{(k+1)} = [\lambda_i^{(k)} + \beta_k (C_i(\pi_k) - d_i)]^+$, where β_k is the step size and $[x]^+ = \max\{0, x\}$ is the projection onto the dual space $\lambda \geq 0$. By comparison, CPO derives the dual variable $\lambda^{(k+1)}$ by solving an optimization problem from scratch in order to enforce the constraints in every iteration.

RL Encouraging Safety: CMDP Framework

- Benchmark: Safety Gym

Benchmarking Safe Exploration in Deep Reinforcement Learning

Alex Ray* OpenAI
Joshua Achiam* OpenAI
Dario Amodei OpenAI

SG18	Return $\bar{J}_r$	Violation $\bar{M}_c$	Cost Rate $\bar{\rho}_c$
PPO	1.0	1.0	1.0
PPO-Lagrangian	0.24	0.026	**0.245**
TRPO	**1.094**	1.132	1.004
TRPO-Lagrangian	0.331	**0.018**	0.265
CPO	0.784	0.593	0.646

Table 1: Normalized metrics from the conclusion of training averaged over the SG18 slate of environments and three random seeds per environment.

Thank You!